PROBABILITY THEORY AND LANGUAGE MODELS

A step-by-step explanation of the uploaded Python program

Learning objective: Understand how a computer counts words and word sequences, then uses those counts to calculate the probability of a sentence.

1. What is a language model?

A language model is a system that learns patterns in language and estimates how likely a word or sequence of words is. For example, after observing sentences such as “I love machine learning” and “I love deep learning”, a model can learn which words occur often and which words tend to follow one another.

A **corpus** is the collection of text used to build the model. This program uses a small corpus of five sentences.

2. Import Counter

```
from collections import Counter
```

Counter is a Python utility that counts how often items occur. For example, Counter(["apple", "banana", "apple"]) records apple: 2 and banana: 1. In this program, it counts individual words, pairs of words and groups of three words.

3. Create the sample corpus

```
corpus = [
    "I love machine learning",
    "I love deep learning",
    "I study machine learning",
    "I study artificial intelligence",
    "Machine learning is interesting"
]
```

The five sentences form the training corpus. The model learns only from the text supplied here; it does not automatically know all English grammar or meaning.

4. Tokenization: convert sentences into words

```
tokenized_corpus = []

for sentence in corpus:
    words = sentence.lower().split()
    tokenized_corpus.append(words)
```

sentence.lower() converts letters to lowercase so that “Machine” and “machine” are treated as the same token. **split()** separates a sentence at whitespace. **append()** adds the resulting word list to tokenized_corpus. This simple process is called tokenization.

```
['i', 'love', 'machine', 'learning']
['i', 'love', 'deep', 'learning']
['i', 'study', 'machine', 'learning']
['i', 'study', 'artificial', 'intelligence']
['machine', 'learning', 'is', 'interesting']
```

5. Unigram: count individual words

```
unigram_counts = Counter()

for sentence in tokenized_corpus:
    unigram_counts.update(sentence)

total_words = sum(unigram_counts.values())
```

A **unigram** considers one word at a time. It does not use the surrounding words. update(sentence) adds the words from each sentence to the counter. sum(unigram_counts.values()) gives the total number of word tokens.

Word	Frequency
i	4
love	2
machine	3
learning	4
deep	1
study	2
artificial	1
intelligence	1
is	1
interesting	1
TOTAL	**20**

Unigram probability: P(w) = Count(w) / Total number of words. For example, P(i) = 4/20 = 0.2 and P(love) = 2/20 = 0.1.

6. Bigram: count consecutive pairs of words

```
bigram_counts = Counter()

for sentence in tokenized_corpus:
    for i in range(len(sentence) - 1):
        bigram = (sentence[i], sentence[i+1])
        bigram_counts[bigram] += 1
```

A **bigram** considers two consecutive words. The sentence “I love machine learning” produces three pairs: (I, love), (love, machine) and (machine, learning). len(sentence) - 1 ensures that i+1 does not go beyond the final word.

Bigram probability: P(w■ | w■) = Count(w■, w■) / Count(w■). It means the probability of the second word given the first word. In this corpus, “I” occurs four times and “I love” occurs twice, so P(love | I) = 2/4 = 0.5.

Bigram	Frequency
i, love	2
love, machine	1
machine, learning	3
love, deep	1
deep, learning	1
i, study	2
study, machine	1
study, artificial	1
artificial, intelligence	1
learning, is	1
is, interesting	1

7. Trigram: count consecutive groups of three words

```
trigram_counts = Counter()
```

```
for sentence in tokenized_corpus:
    for i in range(len(sentence) - 2):
        trigram = (
            sentence[i],
            sentence[i+1],
            sentence[i+2]
        )
        trigram_counts[trigram] += 1
```

A **trigram** considers three consecutive words. “I love machine learning” produces (I, love, machine) and (love, machine, learning). The loop uses len(sentence) - 2 because it reads the current word and the next two words.

Trigram probability: P(w■ | w■, w■) = Count(w■, w■, w■) / Count(w■, w■). For example, “I love machine” occurs once and “I love” occurs twice. Therefore P(machine | I, love) = 1/2 = 0.5.

8. Calculate the probability of a test sentence

```
test_sentence = "I love machine learning"
words = test_sentence.lower().split()
```

The program tests the sentence “I love machine learning” against the patterns learned from the corpus. It initializes each probability product to 1.0 and multiplies the probability of each word or sequence.

A. Unigram probability

```
unigram_probability = 1.0

for word in words:
    probability = unigram_counts[word] / total_words
    unigram_probability *= probability
```

The calculation is (4/20) × (2/20) × (3/20) × (4/20) = **0.0006**. Each word is treated independently.

B. Bigram probability

```
bigram_probability = 1.0
bigram_words = ["<s>"] + words
```

The marker <s> means the start of a sentence. The intended sequence is <s> → I → love → machine → learning. The conditional probabilities are multiplied: P(I | <s>) × P(love | I) × P(machine | love) × P(learning | machine).

C. Trigram probability

```
trigram_probability = 1.0
trigram_words = ["<s>", "<s>"] + words
```

Two start markers are used because a trigram uses two previous positions. The terms are P(I | <s>, <s>) × P(love | <s>, I) × P(machine | I, love) × P(learning | love, machine).

9. Display the results

```
print("Word Counts:", unigram_counts)
print("Bigram Counts:", bigram_counts)
print("Trigram Counts:", trigram_counts)
print("Sentence:", test_sentence)
print("UNIGRAM PROBABILITY:", unigram_probability)
print("BIGRAM PROBABILITY:", bigram_probability)
print("TRIGRAM PROBABILITY:", trigram_probability)
```

The final print statements display the learned counts and the three sentence-probability values.

10. Important correction to the uploaded program

The original program adds the start marker <s> when calculating the test sentence probability, but it does not add the same marker when counting bigrams and trigrams in the training corpus. Therefore, the first bigram and trigram counts involving <s> are zero. This makes the original bigram and trigram sentence probabilities zero.

Corrected bigram counting:

```
bigram_counts = Counter()

for sentence in tokenized_corpus:
    sentence = ["<s>"] + sentence
    for i in range(len(sentence) - 1):
        bigram = (sentence[i], sentence[i+1])
        bigram_counts[bigram] += 1
```

Corrected trigram counting:

```
trigram_counts = Counter()

for sentence in tokenized_corpus:
    sentence = ["<s>", "<s>"] + sentence
    for i in range(len(sentence) - 2):
        trigram = (sentence[i], sentence[i+1], sentence[i+2])
        trigram_counts[trigram] += 1
```

After adding these markers during training, the existing probability calculations can use matching start-marker counts.

Corrected results for the test sentence:

Model	Calculation	Probability
Unigram	0.2 × 0.1 × 0.15 × 0.2	0.0006
Bigram	1 × 0.5 × 0.5 × 1	0.25
Trigram	1 × 0.5 × 0.5 × 1	0.25

These values are for this particular five-sentence corpus and this sentence. The program does not use smoothing. The displayed trigram value follows from the observed sequence counts and sentence-start markers.

11. Comparison of the three language models

Model	What it considers	Example from the test sentence
Unigram	One word at a time	I | love | machine | learning
Bigram	Two consecutive words	I love | love machine | machine learning
Trigram	Three consecutive words	I love machine | love machine learning

Remember:

- Unigram: What is the probability of a word?
- Bigram: What is the probability of a word given the previous word?
- Trigram: What is the probability of a word given the previous two words?

12. Applications

Statistical language models can support predictive text, autocomplete, speech recognition, spell checking, machine translation and language-processing systems. They estimate likely word sequences from patterns in training text.

13. Important limitation: zero probabilities and smoothing

A word sequence that never appears in the training corpus receives a count of zero. Without additional treatment, its probability becomes zero and can make the probability of the whole sentence zero. A common solution is **smoothing**, which assigns a small non-zero probability to unseen events. Smoothing is not implemented in the uploaded program or in the corrected calculations above.

14. Suggested execution steps

1. Save the Python file, for example as Probability_Theory_and_Language_Model1.py.
2. Open a terminal or Command Prompt in the folder containing the file.
3. Run: `python Probability_Theory_and_Language_Model1.py`
4. Read the printed unigram, bigram and trigram counts, followed by the three probability values.

End of explanation